

Worked Solutions

Data Analytics

Set A: Ensemble | CSE Semester 3

Companion to da-ensemble.pdf. Question numbers match that paper. Equivalent correct methods are acceptable. These are independent worked solutions, not an official university marking scheme.

Questions 1–5: MCQ answer key, 1 mark each (5 marks total). Questions 6 onward: theory solutions (25 marks total).

1. Null hypothesis

Correct option: (A). The null hypothesis specifies the population parameter: $H_0 : \mu = 0.95$.

2. Standard error

Correct option: (C). $SE = \sigma / \sqrt{n} = 0.02 / \sqrt{36} = 0.02/6$.

3. Normal-test statistic

Correct option: (B). $z = (0.947 - 0.95) / (0.02/6) = -0.90$.

4. Equal-width bins

Correct option: (D). Bin width is $(480 - 180) / 3 = 100$.

5. Equal-frequency bins

Correct option: (B). Each bin contains $12/3 = 4$ observations.

6. Descriptive statistics, preparation and ANOVA

(i) $N = 20$, $\sum x_i = 1350$, $\bar{x} = 67.5$, and the median is 63. Using the population denominator,

$$\sigma^2 = \frac{\sum (x_i - 67.5)^2}{20} = 238.35, \quad \sigma = 15.4386.$$

Pearson's second coefficient is $3(67.5 - 63) / 15.4386 = 0.8744$, indicating positive skewness.

(ii) Combine quantity and unit price to create revenue: $\text{revenue} = \text{quantity} \times \text{unit price}$. This gives a useful derived variable from two existing columns.

(iii) Test $H_0 : \mu_A = \mu_B = \mu_C$ against at least one unequal mean. The group means are 3, 5 and 7; the grand mean is 5.

$$SS_B = 4[(3 - 5)^2 + (5 - 5)^2 + (7 - 5)^2] = 32, \quad SS_W = 4 + 2 + 4 = 10.$$

Source	SS	df	MS	F
Between	32	2	16	14.4
Within	10	9	1.1111	
Total	42	11		

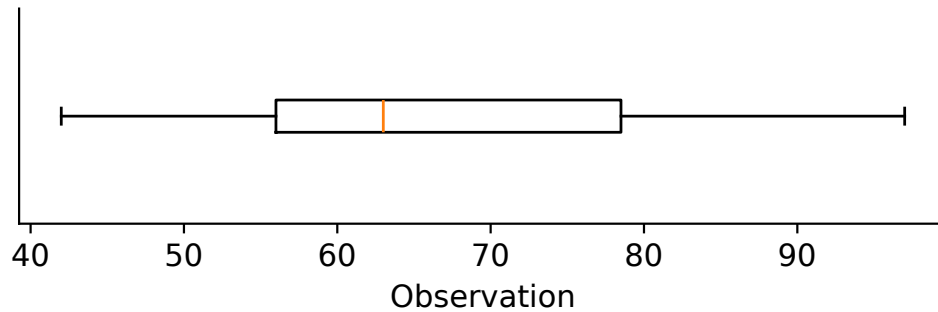
Since $14.4 > 4.256$, reject H_0 at 5%. Mean flavour ratings differ. The test alone does not identify which pairs differ.

7. Box plot and Spearman correlation

(i) Sorted observations:

42, 47, 50, 50, 54, 58, 60, 61, 62, 63, 63, 68, 75, 78, 78, 79, 82, 91, 92, 97.

The five-number summary is $[42, 56, 63, 78.5, 97]$. Here $Q_1 = (54 + 58)/2 = 56$, $Q_3 = (78 + 79)/2 = 78.5$, and $IQR = 22.5$. The fences are 22.25 and 112.25, so there are no outliers.



(ii) Subsets support analysis of relevant cohorts, reduce computational load, and allow separate training and validation sets. Select them appropriately to avoid introducing bias.

(iii) The first two ranks agree. The other ten items have rank differences ± 1 , so $\sum d_i^2 = 10$.

$$r_s = 1 - \frac{6(10)}{12(12^2 - 1)} = \boxed{0.9650}.$$

The interviewers have strong positive rank agreement.

8. Left-tailed normal test

Under the supplied model, $H_0 : \mu = 0.95$ and $H_1 : \mu < 0.95$.

$$z = \frac{0.93 - 0.95}{0.04/\sqrt{50}} = \boxed{-3.5355}.$$

Since $-3.5355 < -1.645$, reject H_0 . The evidence supports a mean accuracy below 95% under this model ($p \approx 0.000203$). The supplied standard deviation is treated as that of accuracy measurements, not inferred as a Bernoulli standard deviation from individual positive/negative results.

9. Vaccination and disease: chi-square test

H_0 : vaccination status and disease status are independent. H_1 : they are associated. Compute each expected count as row total times column total divided by 2000.

Group	Attacked: observed / expected	Not attacked: observed / expected
Vaccinated	31 / 54	469 / 446
Not vaccinated	185 / 162	1315 / 1338

$$\chi^2 = \frac{23^2}{54} + \frac{23^2}{446} + \frac{23^2}{162} + \frac{23^2}{1338} = \boxed{14.6432}.$$

There is one degree of freedom. Since $14.6432 > 3.841$, reject independence. The observed attack rates are $31/500 = 6.2\%$ and $185/1500 = 12.33\%$. Vaccination is associated with a lower attack rate in this sample; the table alone does not establish causation.

10. Two-factor ANOVA with replication

There are $a = 3$ varieties, $b = 4$ fertilizers and $r = 2$ replicates per cell. The grand mean is 8.6667. Variety means are (8.5, 7.25, 10.25) and fertilizer means are (8.1667, 9.5, 6.1667, 10.8333). Use

$$SS_A = br \sum_i (\bar{y}_{i..} - \bar{y})^2, \quad SS_B = ar \sum_j (\bar{y}_{.j.} - \bar{y})^2,$$

$$SS_{AB} = r \sum_{i,j} (\bar{y}_{ij.} - \bar{y}_{i..} - \bar{y}_{.j.} + \bar{y})^2,$$

$$SS_E = \sum_{i,j,k} (y_{ijk} - \bar{y}_{ij.})^2.$$

Source	SS	df	MS	F
Variety	36.3333	2	18.1667	36.3333
Fertilizer	71.3333	3	23.7778	47.5556
Interaction	3.6667	6	0.6111	1.2222
Error	6	12	0.5	
Total	117.3333	23		

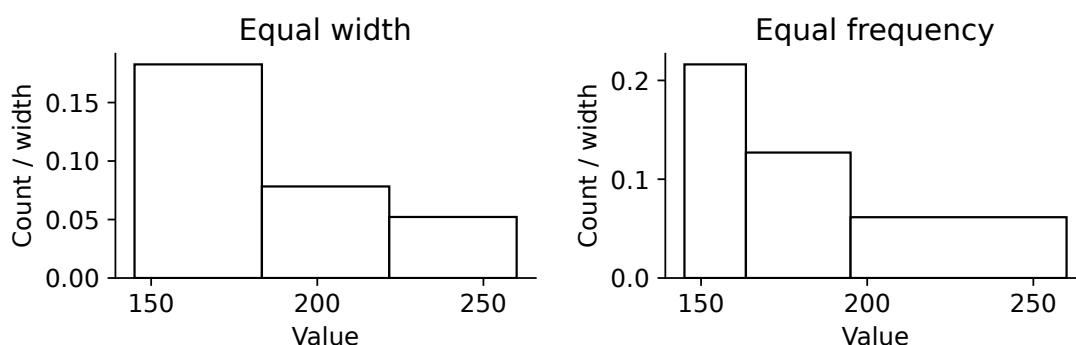
Reject equal variety means because $36.3333 > 3.885$. Reject equal fertilizer means because $47.5556 > 3.490$. Both factors have significant effects at 5% under the supplied assumptions. The interaction is retained in the model; its statistic is shown for completeness. The residual error mean square, rather than the interaction mean square, is the denominator for these fixed-factor tests.

11. Cholesterol binning

Range = $260 - 145 = 115$, so width = $115/3 = 38.3333$.

Method	Bin 1	Bin 2	Bin 3
Equal width	[145,183.333): 7	[183.333,221.667): 3	[221.667,260]: 2
Equal frequency	145,155,160,162	165,172,180,190	200,220,240,260

For equal frequency, use boundaries 145, 163.5, 195 and 260. Equal width better displays the declining concentration across these data and the upper tail. Equal frequency gives balanced groups but requires density heights to make its unequal-width histogram meaningful.



Worked Solutions

Data Structures

Set A: Ensemble | CSE Semester 3

Companion to `ds-ensemble.pdf`. Question numbers match that paper. Equivalent correct methods are acceptable. These are independent worked solutions, not an official university marking scheme.

Questions 1–5: MCQ answer key, 1 mark each (5 marks total). Questions 6 onward: theory solutions (25 marks total).

1. Undo data structure

Correct option: (B). A stack removes the most recently pushed action first (LIFO).

2. First undo

Correct option: (D). C is the most recent edit and is at the top of the stack.

3. Redo stack order

Correct option: (A). The first undo pushes C onto redo; the second pushes B above C.

4. Redo operation

Correct option: (C). Redo pops the top action, B, and puts it back on undo.

5. Edit after undo

Correct option: (B). A new edit starts a new editing branch, so the previously undone future actions are discarded.

6. Nested employee structure

The outer structure stores employee details and the nested `Date` stores joining date. This program assumes a valid calendar date and a one-word name of at most 39 characters. The input-conversion count is checked.

```
#include <stdio.h>
typedef struct { int day, month, year; } Date;
typedef struct {
    int id;
    char name[40];
    Date joined;
} Employee;
int main(void) {
    Employee e;
    /* Input: id, one-word name, day, month, year. */
    if (scanf("%d %39s %d %d %d", &e.id, e.name,
            &e.joined.day, &e.joined.month,
            &e.joined.year) != 5) return 1;
    printf("%d %s %02d/%02d/%04d\n", e.id, e.name,
            e.joined.day, e.joined.month, e.joined.year);
    return 0;
}
```

7. Dynamic matrix and pointer arithmetic

A pointer to an array of three doubles provides contiguous dynamic storage for the three subjects. The expression $*(*(marks + i) + j)$ accesses one mark using pointer arithmetic. Time is $O(n)$ for three subjects and storage is $O(n)$.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main(void) {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 1;
    if ((size_t)n > SIZE_MAX / (3 * sizeof(double))) return 1;
    double (*marks)[3] = malloc((size_t)n * sizeof *marks);
    if (!marks) return 1;
    for (int i = 0; i < n; ++i) {
        double sum = 0;
        for (int j = 0; j < 3; ++j) {
            if (scanf("%lf", *(marks + i) + j) != 1) {
                free(marks); return 1;
            }
            sum += (*(marks + i) + j);
        }
        printf("Student %d average: %.2f\n", i + 1, sum / 3);
    }
    free(marks);
    return 0;
}
```

8. Dynamic string array and concatenation

Allocate two fixed-capacity rows dynamically, then allocate the exact combined length plus one byte for the null terminator. The pointer `dst` advances through the result as each character is copied. Inputs must each fit in 99 characters before their newline.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(void) {
    /* Two rows of up to 99 characters, including spaces. */
    char (*s)[100] = malloc(2 * sizeof *s);
    if (!s) return 1;
    if (!fgets(*s, 100, stdin) || !fgets(*(s + 1), 100, stdin)) {
        free(s); return 1;
    }
    *(*s + strcspn(*s, "\n")) = '\0';
    *(*s + 1) + strcspn(*(s + 1), "\n")) = '\0';
    char *result = malloc(strlen(*s) + strlen(*(s + 1)) + 1);
    if (!result) { free(s); return 1; }
    char *dst = result;
    for (int row = 0; row < 2; ++row)
        for (const char *src = *(s + row); *src; ++src)
            *dst++ = *src;
    *dst = '\0';
    puts(result);
    free(result); free(s);
    return 0;
}
```

9. Playlist insertion and deletion

Append preserves insertion order. A pointer to the current link permits deletion of the head and consecutive short songs without a separate head special case. Deletion and display take $O(n)$ time; this simple append scans to the tail and takes $O(n)$ per insertion. Enter 180 in the demonstration to retain only Evening.

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Song {
    int id, duration;
    char title[80];
    struct Song *next;
} Song;

int append(Song **head, int id, const char *title, int duration) {
    Song *node = malloc(sizeof *node);
    if (!node) return 0;
    node->id = id; node->duration = duration; node->next = NULL;
    snprintf(node->title, sizeof node->title, "%s", title);
    Song **link = head;
    while (*link) link = &(*link)->next;
    *link = node;
    return 1;
}

void removeShort(Song **head, int limit) {
    Song **link = head;
    while (*link) {
        if ((*link)->duration < limit) {
            Song *old = *link;
            *link = old->next;
            free(old);
        } else link = &(*link)->next;
    }
}

void display(const Song *p) {
    for (; p; p = p->next)
        printf("%d %s %d\n", p->id, p->title, p->duration);
}

void clear(Song **head) {
    while (*head) {
        Song *old = *head; *head = old->next; free(old);
    }
}

int main(void) {
    Song *head = NULL;
    if (!append(&head, 1, "Dawn", 120) ||
        !append(&head, 2, "Evening", 240) ||
        !append(&head, 3, "Night", 150)) {
        clear(&head); return 1;
    }
    int limit;
    if (scanf("%d", &limit) != 1) { clear(&head); return 1; }
    removeShort(&head, limit);
    display(head);
    clear(&head);
    return 0;
}
```

10. Recent contacts and traversal

Insert at the head in $O(1)$ time. Forward and reverse printing take $O(n)$ time. The recursive reverse printer uses $O(n)$ call-stack space and does not change the list. Each call adds a contact record; moving an already present contact would additionally require locating and unlinking its previous node.

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Contact {
    int id;
    char name[40];
    struct Contact *next;
} Contact;
int addRecent(Contact **head, int id, const char *name) {
    Contact *p = malloc(sizeof *p);
    if (!p) return 0;
    p->id = id;
    snprintf(p->name, sizeof p->name, "%s", name);
    p->next = *head; *head = p;
    return 1;
}
void forward(const Contact *p) {
    for (; p; p = p->next) printf("%d %s\n", p->id, p->name);
}
void reversePrint(const Contact *p) {
    if (!p) return;
    reversePrint(p->next);
    printf("%d %s\n", p->id, p->name);
}
```

11. Infix to postfix with precedence functions

Pop while in-stack precedence is at least incoming precedence. Giving incoming exponentiation precedence 6 and in-stack exponentiation precedence 5 keeps equal exponentiation operators on the stack, producing right associativity. Equal-precedence addition, subtraction, multiplication and division pop, producing left associativity. This C99 implementation handles valid expressions with single-character operands and optional parentheses.

```
#include <ctype.h>
#include <stddef.h>
#include <string.h>
int InStackPrecedence(char op) {
    switch (op) {
        case '+': case '-': return 1;
        case '*': case '/': return 3;
        case '^': return 5;
        case '(': return 0;
        default: return -1;
    }
}
int IncomingPrecedence(char op) {
    switch (op) {
        case '+': case '-': return 1;
        case '*': case '/': return 3;
        case '^': return 6; /* Do not pop an existing '^'. */
        case '(': return 7;
        default: return -1;
    }
}
/* Valid expressions, single-character operands; output has
   space for strlen(infix)+1 bytes. Parentheses are supported. */
void Infix_Postfix(char infix[], char postfix[]) {
    char stack[strlen(infix) + 1];
    size_t top = 0, out = 0;
    for (size_t i = 0; infix[i]; ++i) {
        char c = infix[i];
        if (isspace((unsigned char)c)) continue;
        if (isalnum((unsigned char)c)) postfix[out++] = c;
        else if (c == '(') stack[top++] = c;
        else if (c == ')') {
            while (top && stack[top - 1] != '(')
                postfix[out++] = stack[--top];
            if (top) --top;
        } else {
            while (top && stack[top - 1] != '(' &&
                InStackPrecedence(stack[top - 1]) >=
                IncomingPrecedence(c))
                postfix[out++] = stack[--top];
            stack[top++] = c;
        }
    }
    while (top) postfix[out++] = stack[--top];
    postfix[out] = '\0';
}
```

For A^B^C , the output is $ABC^{\wedge\wedge}$. For $A-B-C$, it is $AB-C-$. Each token is pushed and popped at most once, so time and auxiliary space are $O(n)$.

12. Duplicate parentheses

Here duplicate means a redundant pair enclosing no operator at its own nesting level, including an extra wrapper around a parenthesized expression. The input is assumed syntactically valid and balanced. Pop a closed group, check whether an operator occurred, and reduce a useful group to one operand marker. Time and stack space are $O(n)$.

```
#include <stdbool.h>
#include <ctype.h>
#include <string.h>
/* Input is a syntactically valid, balanced expression.
   Redundant pair means a pair enclosing no operator at its
   own nesting level: (a), ((a+b)), and () are detected. */
bool isDuplicate(char s[]) {
    char stack[strlen(s) + 1];
    size_t top = 0;
    for (size_t i = 0; s[i]; ++i) {
        char c = s[i];
        if (isspace((unsigned char)c)) continue;
        if (c != ')') stack[top++] = c;
        else {
            bool hasOperator = false;
            while (top && stack[top - 1] != '(') {
                char v = stack[--top];
                if (strchr("+-*/^", v)) hasOperator = true;
            }
            if (top) --top;
            if (!hasOperator) return true;
            stack[top++] = 'x'; /* One reduced subexpression. */
        }
    }
    return false;
}
```

Examples: ((a+b)) returns true; (a+b) returns false; (a) returns true under the stated convention.

13. Activation records and LIFO

Calls push frames in this order: main, fun2, fun1. Returns remove frames in the reverse order: fun1, fun2, main. The output is:

```
Inside fun1
Inside fun2
Inside main
```

A frame holds the return address, parameters, local variables and saved execution state as required by the calling convention. A queue would remove the oldest caller first, although that caller is suspended awaiting its callee, so it does not represent normal nested call/return order.

Worked Solutions

Data Communication and Computer Networks

Set A: Ensemble | CSE Semester 3

Companion to dccn-ensemble.pdf. Question numbers match that paper. Equivalent correct methods are acceptable. These are independent worked solutions, not an official university marking scheme.

Questions 1–5: MCQ answer key, 1 mark each (5 marks total). Questions 6 onward: theory solutions (25 marks total).

1. Layered headers

Correct option: (B). The total is $900 + 5(20) = 1000$ bytes.

2. Header percentage

Correct option: (D). The fraction is $100/(900 + 100) = 0.10 = 10\%$.

3. CRC degree

Correct option: (C). The leading term is x^4 , so the generator has degree 4.

4. CRC appended zeros

Correct option: (A). Append as many zeros as the generator degree, which is 4.

5. Modulo-2 subtraction

Correct option: (B). Modulo-2 subtraction is XOR, with no carry or borrow.

6. ASK, FSK and PSK

ASK changes carrier amplitude, FSK changes carrier frequency, and PSK changes carrier phase. Binary FSK commonly uses two separated frequencies and needs more bandwidth than binary PSK under comparable simple pulse shaping. ASK and BPSK can have comparable nominal bandwidth for the same bit rate and shaping; exact occupied bandwidth depends on the implementation. ASK is directly vulnerable to amplitude disturbances. Constant-envelope FSK and PSK are less affected by such disturbances, although phase/frequency noise and receiver synchronization still matter.

For a coherent receiver in an approximately additive-noise, bandwidth-limited telephone channel, BPSK is a defensible choice: it uses antipodal signals, offers good bit-error performance at a given energy per bit, and avoids the tone separation of binary FSK. A noncoherent, very simple receiver could instead favor FSK. The noise model and implementation must be stated rather than claiming one scheme is always best.

7. Data rate and bandwidth

Data rate is information bits transmitted per second. Here bandwidth means the span from the lowest to highest retained frequency. The three frequencies are 2, 6 and 10 MHz.

$$R_b = 2f_0 = 2(2 \text{ MHz}) = \boxed{4 \text{ Mbps}}, \quad B = 10 - 2 = \boxed{8 \text{ MHz}}.$$

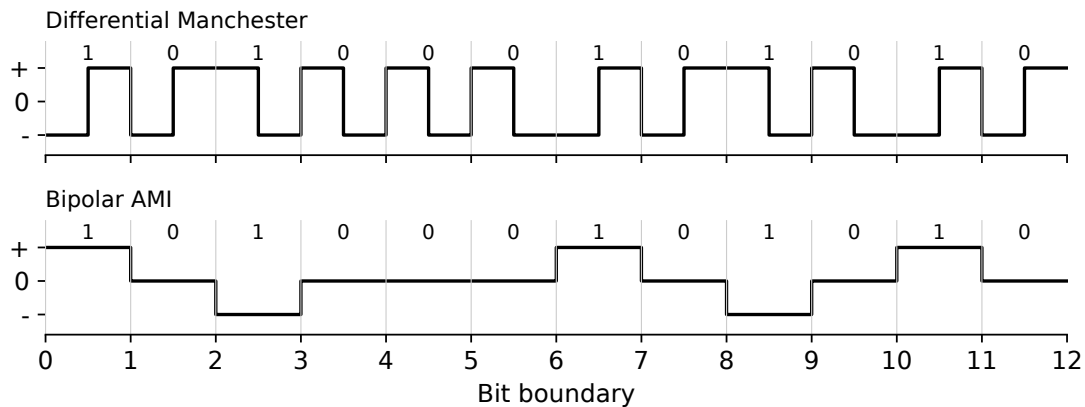
After tripling all frequencies, the components are 6, 18 and 30 MHz:

$$R'_b = \boxed{12 \text{ Mbps}}, \quad B' = \boxed{24 \text{ MHz}}.$$

For this fixed waveform model, both increase by a factor of three. A greater usable channel bandwidth permits faster signal changes, but achievable reliable data rate also depends on noise, signal levels and coding. The two-bits-per-cycle assumption is essential; it is not inferred from a sinusoid alone.

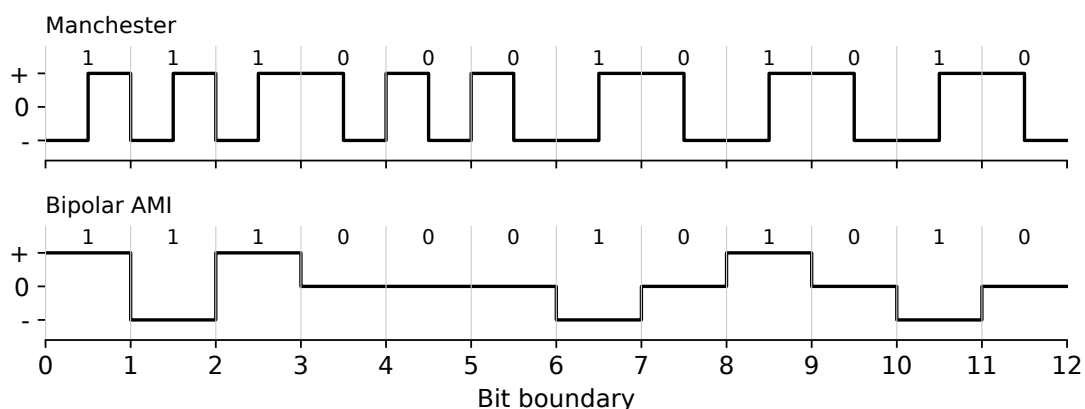
8. Differential Manchester and AMI

Start at a negative level. For differential Manchester, a 0 causes a transition at the start of its bit, a 1 does not, and every bit has a mid-bit transition. For AMI, zeros have zero level and successive ones alternate positive and negative, starting positive because the prior nonzero pulse was negative. The diagram labels each bit boundary.



9. Manchester and AMI

Use low-to-high for Manchester 1 and high-to-low for Manchester 0. AMI ones alternate polarity, with the first one positive; zeros are at zero level.



10. Hub, switch and router

The hub repeats the Sales signal out its other ports without inspecting a destination address. HR and the attached switch therefore see it. The switch learns the source MAC address on the ingress port and forwards the frame only to the known IT destination port. If that MAC is unknown, it floods within the VLAN except back to the ingress port. A destination learned on the ingress port is filtered.

If Sales and IT are in the same IP subnet and VLAN, their local traffic need not pass through the Internet router. If they are in different subnets/VLANs, the sender uses its default gateway and the router must route between those networks. A router forwards according to destination IP and installs a new link-layer header on the outgoing link. Merely being connected to the Internet does not make it part of every local transfer.

11. ARQ retransmission counts

Initially frames 0, 1 and 2 are transmitted, with frame 1 lost.

Protocol	Receiver handling of 2	Retransmitted	Total sends	Useful/total
Go-back-N	Discard	1, 2	5	3/5 = 0.60
Selective repeat	Buffer	1	4	3/4 = 0.75

Go-back-N has already accepted frame 0; on timeout it resends the outstanding suffix 1 and 2. Selective repeat acknowledges and buffers frame 2, then needs only frame 1 before delivering 1 and 2 in order. Counts use the clarified three-frame scenario printed in this paper, not the older ambiguous repeated-loss wording.

12. Propagation-limited stop-and-wait time

Number of frames = $10^6/2000 = 500$.

$$t_p = \frac{5000 \times 10^3}{2 \times 10^8} = 0.025 \text{ s}, \quad RTT = 0.05 \text{ s}.$$

Counting one complete round trip for each frame, including the final ACK, gives

$$T = 500(0.05) = \boxed{25 \text{ s}}.$$

Transmission, ACK serialization and processing times are neglected as requested.

13. Collision-detection frame length

$$t_p = \frac{1500}{3 \times 10^8} = 5 \mu\text{s}, \quad L_{\min} = R(2t_p) = 10^8(10 \times 10^{-6}) = \boxed{1000 \text{ bits}}.$$

This is 125 bytes under the question's propagation-only model. The sender must still be transmitting when a far-end collision can propagate back.

Worked Solutions

Digital Systems and Computer Organization

Set A: Ensemble | CSE Semester 3

Companion to `dsc0-ensemble.pdf`. Question numbers match that paper. Equivalent correct methods are acceptable. These are independent worked solutions, not an official university marking scheme.

Questions 1–5: MCQ answer key, 1 mark each (5 marks total). Questions 6 onward: theory solutions (25 marks total).

1. Signed range

Correct option: (B). For n bits, the range is -2^{n-1} to $2^{n-1} - 1$, giving -16 to 15 .

2. Negative representation

Correct option: (D). Invert `01001` to get `10110`, then add 1 to obtain `10111`.

3. Signed overflow

Correct option: (C). $11 + 7 = 18$ exceeds the maximum signed value 15 ; the other results lie in range.

4. PLA arrays

Correct option: (A). A PLA has programmable AND and OR arrays for constructing and combining product terms.

5. Shared product term

Correct option: (B). AB appears in both outputs and can feed both OR connections.

6. POS minimization and NOR implementation

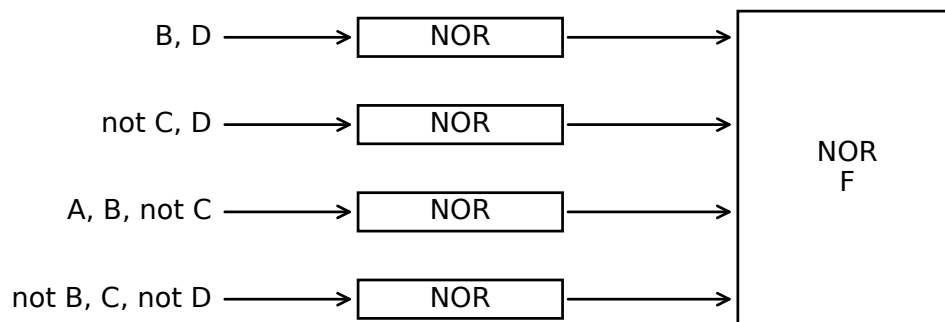
Use a K-map with rows AB and columns CD in Gray order 00, 01, 11, 10. X denotes a don't-care.

AB / CD	00	01	11	10
00	0	1	0	X
01	1	0	1	0
11	1	X	1	X
10	0	1	1	0

Group zeros with useful don't-cares: $\{0, 2, 8, 10\}$, $\{2, 6, 10, 14\}$, $\{2, 3\}$ and $\{5, 13\}$. These give

$$F = (B + D)(\bar{C} + D)(A + B + \bar{C})(\bar{B} + C + \bar{D}).$$

Each first-level NOR gives the complement of one sum. A final NOR of those four outputs produces their uncomplemented product by De Morgan's law. Generate $\bar{B}$, $\bar{C}$, $\bar{D}$ with tied-input NOR gates: $\bar{X} = \text{NOR}(X, X)$.



7. Three-bit magnitude comparator

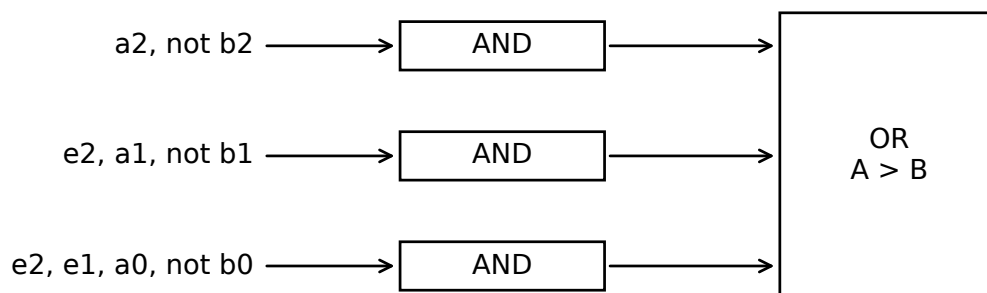
Define $e_i = a_i \text{ XNOR } b_i$. The most significant unequal bit decides the ordering:

$$G = a_2\bar{b}_2 + e_2a_1\bar{b}_1 + e_2e_1a_0\bar{b}_0,$$

$$E_q = e_2e_1e_0,$$

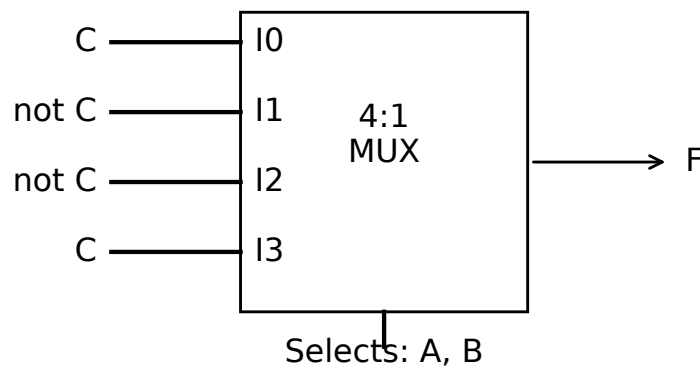
$$L = \bar{a}_2b_2 + e_2\bar{a}_1b_1 + e_2e_1\bar{a}_0b_0.$$

Generate each e_i with an XNOR gate and each complemented input with NOT. The following AND-OR network implements G. Implement L with the same network after exchanging a and b. A three-input AND of the XNOR outputs implements E_q . These three networks together form the comparator.



8. Three-input XOR using a multiplexer

With A and B as select bits (A most significant), $AB = 00, 01, 10, 11$ produces $C, \bar{C}, \bar{C}, C$, respectively. Thus $I_0 = C, I_1 = \bar{C}, I_2 = \bar{C}, I_3 = C$. One inverter supplies both complemented-C inputs.



9. Enabled synchronous up/down counter

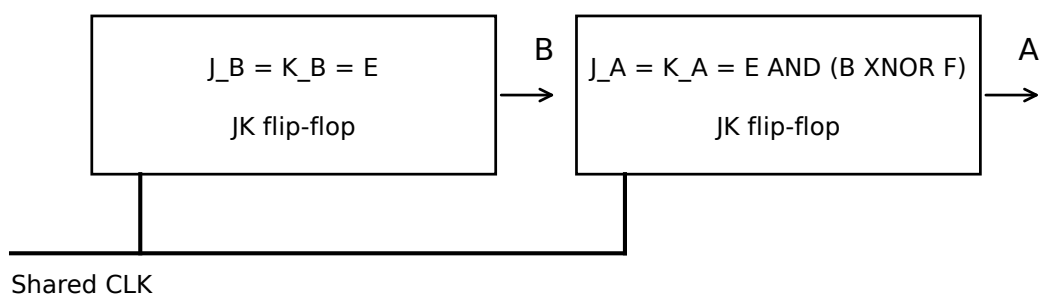
Let A be the most significant state bit and B the least significant. The state table is:

Present AB	E=0, either F	E=1, F=1	E=1, F=0
00	00	01	11
01	01	10	00
10	10	11	01
11	11	00	10

A JK flip-flop toggles when $J = K = 1$ and holds when $J = K = 0$. B toggles whenever enabled. A toggles on an upward carry ($B=1, F=1$) or downward borrow ($B=0, F=0$).

$$J_B = K_B = E, \quad J_A = K_A = E(BF + \bar{B}\bar{F}).$$

Equivalently, the second excitation is E AND (B XNOR F). Connect both flip-flops to the same active clock edge and use the following excitation network.

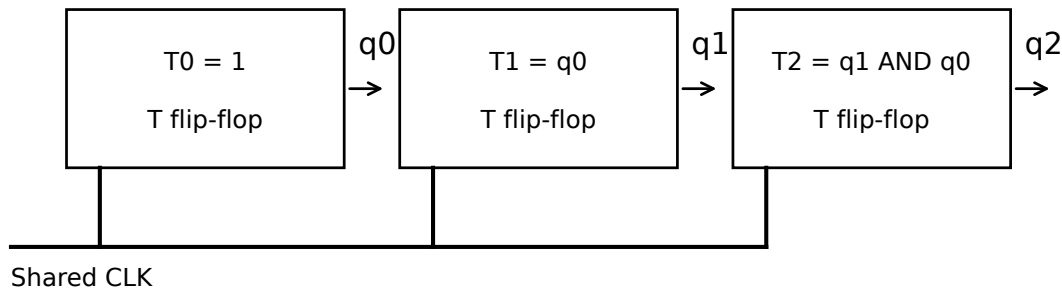


10. Three-bit synchronous T counter

For $q_2q_1q_0$, the least significant bit toggles every clock; bit 1 toggles when $q_0 = 1$; bit 2 toggles when $q_1q_0 = 11$.

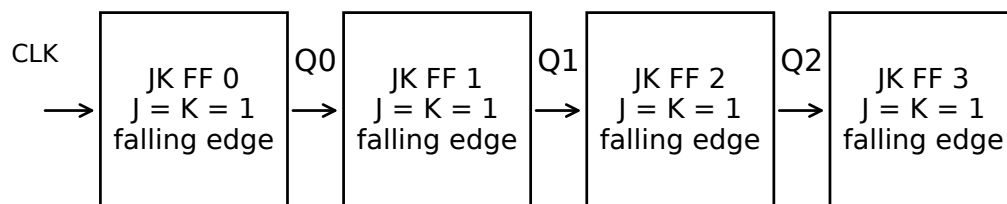
$$T_0 = 1, \quad T_1 = q_0, \quad T_2 = q_1q_0.$$

All three T flip-flops use the same clock. Feed q_1, q_0 through an AND gate to T_2 . The sequence is 000, 001, 010, 011, 100, 101, 110, 111, 000.



11. Four-bit asynchronous ripple counter

Use four falling-edge-triggered JK flip-flops with $J=K=1$. Apply the external clock to stage 0 and connect each stage's Q to the next stage's clock, as shown. Clear all stages to 0 before counting.



The output $q_3q_2q_1q_0$ counts 0000 through 1111 and repeats. Output frequencies are $f/2, f/4, f/8, f/16$. Since changes propagate stage by stage, transitions such as 0111 to 1000 pass through intermediate states. Decoding before the ripple settles can produce glitches. The falling-edge convention is necessary for this Q-to-clock up-counter connection.

12. Eight-bit ring counter

The question leaves reset details open; choose an active-high synchronous reset to load a single 1. This makes the initial state deterministic. Each clock rotates left, including the wrap from bit 7 to bit 0.

```
module ring8(input wire clk, input wire reset,
             output reg [7:0] q);
    always @(posedge clk) begin
        if (reset) q <= 8'b00000001;
        else q <= {q[6:0], q[7]};
    end
endmodule
```

The cycle is 00000001, 00000010, 00000100, 00001000, 00010000, 00100000, 01000000, 10000000, then 00000001. Reset must be asserted to enter this one-hot cycle; an all-zero state would otherwise remain zero.

13. Byte and word addressing

A memory address identifies an addressable unit of storage. An ISA defines how effective addresses are formed and how load/store instructions interpret them. In byte addressing, consecutive addresses select consecutive bytes. A 32-bit word therefore spans four byte addresses, and consecutive aligned words differ by 4. In word addressing, consecutive addresses select entire words, so consecutive words differ by 1. For 2^a addressable units, capacity is 2^a bytes in the byte-addressed case and $2^a w$ bytes for w-byte word addressing. Word size and address width are separate concepts.

Worked Solutions

Discrete Mathematical Structures

Set A: Ensemble | CSE Semester 3

Companion to dms-ensemble.pdf. Question numbers match that paper. Equivalent correct methods are acceptable. These are independent worked solutions, not an official university marking scheme.

Questions 1–5: MCQ answer key, 1 mark each (5 marks total). Questions 6 onward: theory solutions (25 marks total).

1. Complement conditions

Correct option: (B). A complement joins with the element to give 1 and meets with it to give 0.

2. Complement uniqueness

Correct option: (D). Complement existence is a Boolean-algebra property, and distributivity gives uniqueness.

3. Boolean simplification

Correct option: (A). Distribution gives $(a \vee \neg a) \wedge (a \vee b) = 1 \wedge (a \vee b) = a \vee b$.

4. Disjunctive normal form

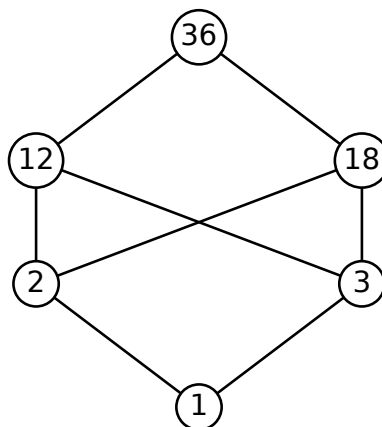
Correct option: (C). $(x \wedge y) \vee (\neg x \wedge z)$ is an OR of two AND terms of literals.

5. Conjunctive normal form

Correct option: (B). $(x \vee y) \wedge (\neg x \vee z)$ is an AND of two OR clauses of literals.

6. Divisibility poset and antichains

The cover relations are $(1, 2), (1, 3), (2, 12), (2, 18), (3, 12), (3, 18), (12, 36), (18, 36)$. They determine the Hasse diagram below; line crossings without a labelled vertex are not additional elements.



This poset is **not a lattice**. The common upper bounds of 2 and 3 are 12, 18 and 36. The two minimal common upper bounds 12 and 18 are incomparable, so there is no least upper bound. Equivalently, 12 and 18 have no greatest lower bound in the set: 2 and 3 are incomparable maximal common lower bounds. It is not valid to import gcd 6 into the set, since 6 is absent.

The complete list of antichains with more than one element is $\{2, 3\}, \{12, 18\}$. Every other distinct pair is comparable, so there are no larger antichains.

7. Bounded item selection

Let $u = A - 1$, $v = B - 3$, $w = C - 5$. Then $u + v + w = 6$, with $0 \leq u \leq 3$, $0 \leq v \leq 4$, $0 \leq w \leq 5$. The unrestricted nonnegative count is $\binom{8}{2} = 28$.

Subtract violations: $u \geq 4$ contributes $\binom{4}{2} = 6$, $v \geq 5$ contributes $\binom{3}{2} = 3$, and $w \geq 6$ contributes $\binom{2}{2} = 1$. Intersections are impossible because their minimum sums exceed 6. Thus

$$28 - 6 - 3 - 1 = 18.$$

Items of the same type are indistinguishable, so this counts feasible triples of quantities, not arrangements.

8. Partition bijection

For positive n , let λ be a partition of n with every part at least 3. In its Ferrers diagram, the first three columns have the same height k . Its conjugate partition $\mu = \lambda'$ therefore satisfies $\mu_1 = \mu_2 = \mu_3 = k$. Define a new partition

$$\nu = (\mu_1 + 2, \mu_2 + 1, \mu_3, \mu_4, \dots).$$

Its total is $n + 3$, and its three largest parts are $k + 2, k + 1, k$, which are consecutive.

Conversely, if a partition of $n + 3$ has largest parts $k + 2, k + 1, k$, subtract 2 from the first part and 1 from the second. The resulting partition has three equal largest parts k and total n . Conjugating it gives a partition of n in which every row has at least three cells, hence every part is at least 3. These constructions undo each other, proving the claimed equality. For $n=1$ or 2 , both sets are empty.

9. The 75th Fike permutation

Use the original descending-exchange Fike convention, with the original arrangement 01234 counted as permutation 1. Enumerate (d_2, d_3, d_4, d_5) with $d_k = k, k - 1, \dots, 1$, and d_5 varying fastest. For each tuple, start from 01234 and exchange positions k and d_k in increasing k order, using one-based positions.

The zero-based index is $75 - 1 = 74$. The block weights for these four choices are 60, 20, 5 and 1:

$$74 = 1(60) + 0(20) + 2(5) + 4(1).$$

Thus $(d_2, d_3, d_4, d_5) = (2 - 1, 3 - 0, 4 - 2, 5 - 4) = (1, 3, 2, 1)$.

Exchange	Arrangement
Start	01234
Positions 2 and 1	10234
Positions 3 and 3	10234
Positions 4 and 2	13204
Positions 5 and 1	43201

Hence the answer under this convention is **43201**.

Convention reference: J. S. Rohl, "Programming Improvements to Fike's Algorithm for Generating Permutations," The Computer Journal 19(2), 156–159 (1976), Figure 1 and recursive procedure. <https://doi.org/10.1093/comjnl/19.2.156>. The convention is stated explicitly because Fike order is not lexicographic order.

10. Regular self-complementary graph

Assume a nonempty simple graph. If G is r -regular on n vertices, its complement is $(n - 1 - r)$ -regular. Isomorphism to its complement implies

$$r = n - 1 - r, \quad r = (n - 1)/2.$$

Therefore n is odd. The handshake lemma says $nr = 2|E|$ is even. Since n is odd, r must be even; write $r = 2t$. Substitution gives $n = 2r + 1 = \boxed{4t + 1}$, with $t \geq 0$.

11. Minimum degree forces connectivity

Suppose G is disconnected, and a component has s vertices. A vertex in that component has degree at most $s - 1$, so

$$s - 1 \geq \delta(G) \geq (p - 1)/2, \quad s \geq (p + 1)/2.$$

Every component must satisfy this bound. Two components would already contain at least $p+1$ vertices, contradicting the total p . Hence G is connected. This argument assumes the usual simple-graph setting; a one-vertex graph is connected directly.

12. Dijkstra on the directed source network

Respect arrow direction. The outgoing edges are: $v_1 \rightarrow v_2 : 4$, $v_1 \rightarrow v_3 : 5$; $v_2 \rightarrow v_1 : 6$, $v_2 \rightarrow v_3 : 4$, $v_2 \rightarrow v_5 : 8$; $v_3 \rightarrow v_2 : 6$, $v_3 \rightarrow v_4 : 2$, $v_3 \rightarrow v_6 : 3$; $v_4 \rightarrow v_5 : 7$; $v_5 \rightarrow v_1 : 3$, $v_5 \rightarrow v_4 : 2$; $v_6 \rightarrow v_3 : 5$, $v_6 \rightarrow v_4 : 4$. The table records tentative distances after relaxing outgoing edges of each settled vertex.

Settled	d_1	d_2	d_3	d_4	d_5	d_6
Initial	0	∞	∞	∞	∞	∞
v_1	0	4	5	∞	∞	∞
v_2	0	4	5	∞	12	∞
v_3	0	4	5	7	12	8
v_4	0	4	5	7	12	8
v_6	0	4	5	7	12	8
v_5	0	4	5	7	12	8

Destination	Shortest path	Length
v_1	v_1	0
v_2	$v_1 \rightarrow v_2$	4
v_3	$v_1 \rightarrow v_3$	5
v_4	$v_1 \rightarrow v_3 \rightarrow v_4$	7
v_5	$v_1 \rightarrow v_2 \rightarrow v_5$	12
v_6	$v_1 \rightarrow v_3 \rightarrow v_6$	8